

SIH26132 — Market Linkage & Price Discovery

Complete Technical Execution Plan & Antigravity Development Blueprint

Purpose: Master reference for implementing the complete farmer/FPO market-intelligence, forecasting, buyer-linkage and transaction platform.

USP: We don't optimize for the highest price. We optimize for the farmer's highest **NET REALIZATION**.

1. Product Definition

- **Goal:** Help farmers/FPOs decide **where, when and to whom to sell** for the best realistic expected earnings.
- **Inputs:** crop, quantity, location, harvest timing, quality, storage availability; current market data; historical prices; buyer demand; logistics; procurement information.
- **Outputs:** current price comparison, price forecast, buyer matches, expected net realization and SELL NOW / WAIT / BEST BUYER recommendation, followed by transaction tracking.

2. Complete Product Workflow

Farmer/FPO Input → Market Intelligence → Price Forecast → Selling Opportunities → Bulk Opportunity? → Buyer Matching → Digital Offer → Logistics → Expected Net Realization → Decision → Deal → Delivery → Quality/Quantity Confirmation → Payment → Actual Net Realization → Transaction Feedback.

3. Five Core Engines

- **Market Intelligence Engine:** current/historical market prices, trends, arrivals where available, demand and procurement pathways.
- **AI Forecasting Engine:** learns historical patterns and forecasts short-term price movement.
- **Decision Engine:** combines forecast with economics and constraints to recommend SELL / WAIT / BEST BUYER.
- **Buyer & Bulk Engine:** matches individual lots and aggregates compatible small lots for larger demand.
- **Logistics & Net Realization Engine:** estimates movement costs and calculates what the farmer actually expects to retain.

4. Historical vs Current Data Architecture

- **Historical Kaggle data:** used for exploration, training, feature engineering, backtesting and model evaluation.
- **Current data.gov.in API:** used for latest market observations and live inference. Current observations can be cached/stored in PostgreSQL with ingestion timestamps.
- **Important:** The model is trained beforehand. A farmer request does not retrain the model. It fetches/uses the latest available data, generates features, runs the saved model and then executes the decision engine.

5. Data Pipeline

- Raw CSV → schema inspection → cleaning → standardization → duplicate/missing-value handling → chronological sorting → processed dataset → feature engineering → training dataset.
- Current API → request/filter → validate response → normalize → timestamp → upsert/cache → current market service → forecast/decision services.
- **Keep raw data unchanged.** Maintain processed/versioned datasets separately.

6. ML Model Plan

- **Target:** short-term future modal/market price for a selected commodity/market context, subject to the actual dataset.
- **Features:** historical prices, lag-1/3/7/14/30, 7/14/30-day moving averages, recent change, volatility, market/district/state, commodity/variety/grade, month/season and reliable arrival-volume features if present.

- **Models:** begin with moving-average baseline, then Linear Regression, Random Forest and XGBoost. Select based on actual validation performance.
- **Output:** forecast range + trend + confidence/uncertainty indicator, not a guaranteed exact price.

7. ML Training / Validation / Testing

- Use chronological splits: older data → training; later data → validation; latest unseen historical period → test.
- Evaluate using MAE, RMSE and optionally MAPE/directional accuracy.
- **Never randomly split time-series data** when that would create future-information leakage.
- Do not fabricate accuracy in the PPT. Use measured results only.

8. Forecast Service

- Production flow: Farmer input + latest current observations + required historical features → preprocessing identical to training → saved model → forecast → decision engine.
- Record model version, prediction timestamp, input context and forecast output where useful for auditability.

9. SELL / WAIT Decision Engine

- ML forecasts price; the decision engine recommends action.
- Inputs: current price, forecast movement, storage availability/cost, liquidity need, buyer availability, transport cost and risk/uncertainty.
- Waiting should be recommended only when expected future benefit reasonably compensates for storage and waiting costs/risk.

10. Net Realization Engine

- **Expected Net Realization = Expected Selling Revenue – Transport – Storage – Handling – Other Applicable Costs.**
- Example: $100Q \times \$2,580 = \$2,58,000$ gross; subtract $\$1,500$ transport + $\$500$ storage + $\$300$ handling = $\$2,55,700$ expected net.
- The system ranks selling options by economic outcome, not headline price alone.

11. Buyer Matching

- Hard filters: crop/commodity, required quality/grade, quantity, delivery window and other mandatory requirements.
- Scoring: price, quantity compatibility, quality fit, distance, delivery fit and buyer reliability.
- Return ranked buyers with an explanation of why each match is suitable.

12. Reverse Marketplace

- Buyer posts requirement such as: Wheat, 500Q, Grade A, moisture $\leq 12\%$, delivery within 10 days.
- System searches farmer/FPO lots and determines whether one lot or several compatible lots can fulfill the requirement.
- Value: fragmented supply becomes organized supply for processors, institutional buyers and other large purchasers.

13. Bulk Aggregation

- Example: Farmer A 30Q + B 45Q + C 80Q + D 25Q + E 70Q = 250Q bulk lot.
- Aggregate using crop, variety/quality, location, harvest window and buyer requirements.
- Maintain contribution traceability so each farmer's quantity remains linked to the bulk lot.

14. Quality Matching

- Example buyer: Wheat, Grade A, moisture $\leq 12\%$. A Grade B/14% lot should not be ranked as a strong match.

- Store quality attributes separately and use them as hard constraints or scoring features depending on the buyer requirement.

15. Digital Lot

- Lot ID, crop, quantity, variety, grade, moisture/quality, harvest date, location/collection point and delivery window.
- Lot becomes the standardized object that buyers can evaluate, offer against and later link to a transaction.

16. Digital Offers

- Buyer Offer → Farmer Accept / Reject / Counter → Final Accepted Terms.
- Store offer history, price, quantity, timestamps and parties. Accepted offer creates the transaction.

17. Logistics Engine

- Origin + destination → OpenStreetMap/OSRM route → distance → quantity/vehicle capacity → estimated trips → configurable transport rate → estimated logistics cost.
- For the SIH MVP, estimate transport; do not claim real fleet dispatch unless implemented.

18. Bulk Logistics

- Multiple farms → collection point(s) → consolidated vehicle → buyer.
- Start with simple consolidated cost estimation. Advanced vehicle-routing optimization is optional.

19. Storage Module

- Show nearby storage options with distance, capacity, cost and possible duration.
- Decision engine compares potential price gain from waiting against storage cost and risk.

20. MSP / Procurement

- Treat MSP/procurement as a pathway or benchmark only where crop, location, eligibility and actual procurement availability make it applicable.
- Compare its expected net realization against mandi/private/institutional options. Never claim automatic guaranteed procurement.

21. Buyer Reliability

- Track recorded transactions, on-time payments, delayed payments and disputes.
- Example: 46 on-time payments out of 50 recorded transactions = 92% historical on-time rate.
- Reliability should influence ranking but should not be called government-certified unless there is actual verification.

22. Transaction Lifecycle

- OFFER CREATED → COUNTER OFFER → ACCEPTED → LOT READY → DISPATCHED → DELIVERED → QUALITY/QUANTITY CONFIRMED → PAYMENT PENDING → PAYMENT RECEIVED → COMPLETED.

23. Payment Tracking

- Statuses: PENDING, PARTIAL, PAID, DISPUTED.
- Store transaction ID, agreed amount, received amount and timestamps. Real payment integration is optional for the prototype.

24. Expected vs Actual Net

- Before deal: expected quantity + expected price + estimated costs → Expected Net.

- After delivery: accepted quantity + actual price + actual costs → Actual Net.
- Store variance to understand where estimates differed from reality.

25. Feedback Loop

- Prediction → Recommendation → Transaction → Actual delivery/cost/payment → Expected-vs-Actual comparison → data for better future estimates.
- Model retraining remains a controlled, versioned process; do not silently retrain during farmer requests.

26. Grievance / Dispute

- Record deal ID, agreed price/quantity/quality, actual delivery, payment issue, evidence, timestamps and status.
- Statuses: OPEN → UNDER REVIEW → RESOLVED or REJECTED.

27. Farmer Dashboard

- Primary cards: Current Price, Forecast, Trend, Sell/Wait, Best Expected Net, Best Buyer, Transport, Storage, Buyer Opportunities, Bulk Opportunities, Active Deals and Payment Status.
- Keep farmer-facing language simple. Technical ML information can appear under an explanation/details section.

28. Frontend Architecture

- Use React.js + Vite + Tailwind CSS + Recharts/Chart.js.
- Pages: Login, Dashboard, Market Prices, Forecast, Buyers, Bulk Lots, Logistics, Transactions, Payments, Reports.
- Reusable components: PriceCard, ForecastChart, RecommendationCard, NetRealizationCard, BuyerCard, LotCard, OfferCard, TransactionTimeline, PaymentStatus.
- Centralize API calls in services/api.js.

29. Backend Architecture

- Use Python + FastAPI.
- Layers: routes → services/business logic → database/models; separate ML service/module.
- Services: market_service, buyer_matching, bulk_aggregation, net_realization, decision_engine, logistics_service.

30. ML Code Architecture

- preprocessing.py → features.py → train.py → evaluate.py → predict.py → model_artifacts/.
- Training and inference must share the same feature-generation logic to prevent feature mismatch.

31. PostgreSQL Database

- Core tables: users, farmers, fpos, commodities, markets, market_prices, buyers, buyer_requirements, lots, lot_quality, offers, transactions, logistics, storage_options, procurement_options, payments, grievances, forecasts.
- Relationships: Farmer/FPO → Lot → Quality; Lot → Offers; accepted Offer → Transaction; Transaction → Logistics + Payment + Grievance.

32. Suggested API Endpoints

- GET /api/market/prices; GET /api/market/trends; GET /api/forecast; GET /api/buyers; GET /api/buyers/match; POST /api/lots; GET /api/lots/{id}; POST /api/offers; POST /api/offers/{id}/counter; POST /api/offers/{id}/accept; GET /api/logistics/estimate; POST /api/transactions; GET /api/transactions/{id}; GET /api/payments; GET /api/reports; POST /api/grievances.

33. Full Project Folder Structure

- frontend/src/components, pages, services, hooks, utils; backend/app/{database,models,schemas,routes,services}; backend/ml/{preprocessing.py,features.py,train.py,evaluate.py,predict.py,model_artifacts}; data/raw; data/processed; Docker files; README.

34. Frontend ↔ Backend ↔ ML Flow

- React Dashboard → HTTP/JSON → FastAPI → PostgreSQL + data.gov.in service + buyer/logistics services + Forecast Service → saved ML model → forecast → Decision Engine → recommendation JSON → React.

35. Authentication & Security

- Use JWT and role-based access: FARMER, FPO, BUYER, ADMIN.
- Hash passwords; validate all inputs; use HTTPS in deployment; keep API keys/secrets in environment variables; never expose secrets in frontend code.

36. External API Reliability

- Handle timeout, empty responses, malformed records and rate limits. If safe, use cached recent data with a visible timestamp; otherwise show a clear unavailable state.

37. Testing Strategy

- **Unit:** net realization, buyer scoring, decision rules, feature generation.
- **API:** auth, market, forecast, lots, offers, transactions.
- **ML:** leakage, chronological split, missing data, feature order, model loading.
- **Integration:** farmer input → current data → forecast → buyer → net realization → recommendation → transaction.
- **Scenarios:** high transport, no storage, quality mismatch, no buyer, payment delay, quantity discrepancy and uncertain forecast.

38. MVP Scope

- Must-have: farmer input/login, current prices, historical chart, ML forecast, SELL/WAIT, buyer list/basic matching, net realization, basic logistics, digital lot, offers and transaction tracking.
- Advanced: bulk automation, reverse marketplace, buyer reliability, storage optimization, MSP comparison, payment integration, grievances, expected-vs-actual feedback and advanced routing.

39. Antigravity Development Rule

- Do not build the entire project in one giant prompt. Implement phase by phase. Before modifying code, inspect the existing architecture and preserve working functionality.
- Every task should specify: module, inputs/outputs, API contract, files to change, tests required and how to run/verify the result.

40. Antigravity Phase 0 — Foundation

- Create repository, README, environment files, frontend/backend folders, coding conventions and initial task board. Freeze initial architecture before feature expansion.

41. Antigravity Phase 1 — Frontend

- Build React/Vite UI with mock data: Dashboard, Market Prices, Forecast, Buyers, Bulk Lots, Logistics and Transactions. Make reusable components and responsive layouts.

42. Antigravity Phase 2 — Backend

- Create FastAPI application, configuration, schemas, models, routes, validation and error handling. Add initial API tests.

43. Antigravity Phase 3 — PostgreSQL

- Create migrations/tables/relationships and seed demo data. Verify CRUD and constraints.

44. Antigravity Phase 4 — Integration

- Connect React to FastAPI through api.js. Replace mock data where real APIs exist. Add loading, error and empty states.

45. Antigravity Phase 5 — Real Historical Data + ML

- First inspect the actual CSV files: schema, date coverage, missing values, duplicates, target availability and useful features. Then implement cleaning, feature engineering, chronological split, baselines, XGBoost, evaluation and model artifact saving.

46. Antigravity Phase 6 — Current API + Forecast

- Integrate data.gov.in current data. Cache/store observations with timestamps. Generate live features using the same preprocessing logic and expose /forecast.

47. Antigravity Phase 7 — Decision + Net

- Implement expected net-realization calculations and transparent SELL/WAIT/BEST BUYER logic using current options, forecast, transport, storage and buyer information.

48. Antigravity Phase 8 — Market Linkage

- Implement buyer requirements, matching, reverse marketplace, digital lots, bulk aggregation and logistics estimation.

49. Antigravity Phase 9 — Transactions

- Implement offer/counteroffer, accepted deals, delivery, quality/quantity confirmation, payment status, buyer reliability and transaction history.

50. Antigravity Phase 10 — Advanced + QA

- Add storage/procurement comparison, grievances, expected-vs-actual analysis, feedback data, security audit, end-to-end testing and final demo seed data.

51. Antigravity Prompt Sequence

- **Prompt 1:** Build foundation + UI only; no ML.
- **Prompt 2:** Build FastAPI + PostgreSQL models/API.
- **Prompt 3:** Connect frontend/backend.
- **Prompt 4:** Inspect real CSVs first, then build ML pipeline.
- **Prompt 5:** Integrate saved model into forecast API.
- **Prompt 6:** Build transparent decision + net realization.
- **Prompt 7:** Build buyers/bulk/logistics.
- **Prompt 8:** Build transactions/payment/reliability/grievances.
- **Prompt 9:** Full QA audit; fix broken flows without changing core architecture.

52. Complete Demo Scenario

- 1) Farmer selects crop and enters quantity/location/quality/storage. 2) Latest market data is fetched. 3) Current prices are compared. 4) Historical trend is shown. 5) ML forecast is displayed. 6) Buyers are matched. 7) Bulk opportunity is checked. 8) Logistics is estimated. 9) Expected net realization is calculated. 10) SELL/WAIT/BEST BUYER is recommended. 11) Digital lot is created. 12) Buyer offer/counteroffer occurs. 13) Transaction is created. 14) Delivery and quality/quantity are recorded. 15) Payment status is updated. 16) Actual net realization and variance are shown.

53. PPT Structure

- 1 Title/Team; 2 Problem; 3 Existing Gap; 4 Solution; 5 How It Works; 6 Data Sources; 7 AI Forecast; 8 SELL/WAIT + Net Realization; 9 Buyer + Bulk; 10 Logistics/Storage/MSP; 11 Digital Transaction; 12 Architecture; 13 Technology Stack; 14 Farmer Report; 15 Benefits; 16 USP/Conclusion.

54. Benefits & Final Positioning

- Better price discovery; smarter sell/wait decisions; better potential net realization; more buyer access; bulk selling for small farmers/FPOs; quality visibility; reduced avoidable transaction/logistics costs; buyer/payment transparency; digital records; lower information asymmetry.
- **Final positioning:** “Our platform moves the farmer from price awareness to decision intelligence — predicting where prices may move, identifying suitable buyers, estimating the cost of reaching them, and recommending the selling option with the best expected net realization.”

55. Reference Code Architecture

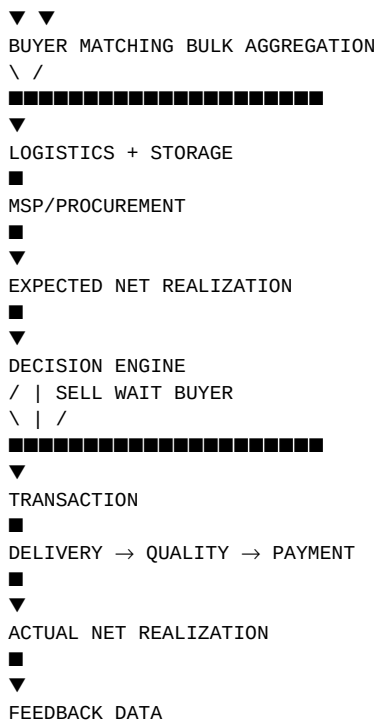
```

agri-market-platform/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/api.js
│   │   ├── hooks/
│   │   ├── utils/
│   │   └── package.json
│   ├── backend/
│   └── app/
│       ├── main.py
│       ├── config.py
│       ├── database/
│       ├── models/
│       ├── schemas/
│       ├── routes/
│       ├── services/
│       └── ml/
│           ├── preprocessing.py
│           ├── features.py
│           ├── train.py
│           ├── evaluate.py
│           ├── predict.py
│           ├── model_artifacts/
│           ├── requirements.txt
│           └── data/
│               ├── raw/
│               ├── processed/
│               ├── docker-compose.yml
│               └── README.md

```

56. Reference System Architecture

[illegible]



57. Critical Implementation Rules

- Historical data trains the model; current API data supports live inference and current market comparison.
- Current observations may be cached/stored, but model retraining is a separate controlled workflow.
- Never randomly split time-series data when it creates future leakage.
- Forecasting and decision-making are separate modules.
- Net realization is the main economic comparison metric.
- Do not fabricate ML accuracy or impact statistics.
- Do not claim MSP procurement is automatically guaranteed.
- Do not recommend buyers that fail hard quality/quantity constraints.
- Do not use every agriculture dataset without checking schema, relevance, coverage and licensing.
- Use the real uploaded CSV schema before finalizing preprocessing.
- Keep API keys/secrets outside frontend code.
- Build and test the MVP end-to-end before advanced features.

58. Final Team Execution Checklist

- 1. Repository + architecture created
- 2. React UI working
- 3. FastAPI working
- 4. PostgreSQL schema working
- 5. Frontend/backend connected
- 6. Historical CSV inspected and cleaned
- 7. Feature pipeline implemented
- 8. Baseline + XGBoost trained
- 9. Chronological test completed

- 10. Model artifact versioned
- 11. data.gov.in current-data integration working
- 12. Forecast API working
- 13. Net realization working
- 14. SELL/WAIT decision working
- 15. Buyer matching working
- 16. Digital lots working
- 17. Bulk aggregation working
- 18. Logistics estimate working
- 19. Offers/counteroffers working
- 20. Transaction lifecycle working
- 21. Payment status working
- 22. Buyer reliability working
- 23. Storage/procurement comparison working or clearly scoped
- 24. Grievance workflow working or clearly scoped
- 25. Expected vs actual net working
- 26. End-to-end demo tested
- 27. PPT metrics replaced with actual measured results